

Club Platform

Modulentwicklung ohne Core

Detaillierte Anleitung · Deutsch
Version 0.6.0 · ABI V3 · Stand 21.09.2026

Bunker Development

Diese PDF wird aus der Markdown-Dokumentation erzeugt. Für das Kopieren längerer Befehle die verlinkte Markdown-Fassung verwenden; lange Codezeilen können im Druck umbrechen. Relative Verweise beziehen sich auf das öffentliche Doku-Repository.

[Markdown-Fassung im Doku-Repository öffnen](#)

Inhalt

1. Was ist ohne Quellcode möglich?	2
2. Vorab mit dem Herausgeber abstimmen	2
3. Öffentliches SDK beziehen	3
4. Das Beispiel auf den drei Systemen bauen	3
5. ABI V3 verstehen	5
6. Eigenes Manifest und eigene Datensätze	6
7. Oberfläche, Daten und Grenzen der Erweiterbarkeit	7
8. Lifecycle und migrationssichere Updates	7
9. Referenzen und Dokumentplatzhalter	8
10. Integration mit einer binären Testinstallation	8
11. Prüfplan für den Entwickler	10
12. Auslieferung über den signierten Modulkatalog	10
13. Fehlerdiagnose	12
14. Übergabe an Bunker Development	12

Entwickleranleitung für das öffentliche SDK · Deutsch · Stand 21.09.2026

Zielstand: Club Platform 0.6.0, Schema 18, Extension ABI V3; abgeglichen mit Produktcommit [5679c9fd868559e12b2140d0da136e2a79879a3b](#). Herausgeber: Bunker Development. Diese Anleitung richtet sich an externe Entwickler, die keinen Zugriff auf das private Produktrepository haben.

PDF: Modulentwicklung ohne Core-Quellcode. Ergänzend: ABI-V3-Referenz, Dokumentplatzhalter, Lizenzierung und Kataloge.

1. Was ist ohne Quellcode möglich?

Ein Modul ist eine native dynamische Bibliothek mit einer kleinen öffentlichen C-Schnittstelle. Es stellt ein JSON-Manifest und Callbacks bereit. Der Host übernimmt Laden, Berechtigungen, Lizenzprüfung, hostverwaltete Datensätze, Installation und Migrationen. Das Modul wird nicht gegen private Core- oder Datenbankbibliotheken gelinkt.

Aufgabe	Benötigte Grundlage
Modul kompilieren	Öffentliche SDK-Header, Beispiel, Compiler, CMake und eigene Bibliotheken
Callback-Logik testen	Eigene Unit-Tests und ABI-Testprogramm; kein privater Core nötig
Oberfläche und Integration prüfen	Separat bereitgestellte Club-Platform-Testinstallation
Lizenzierte Moduloperationen prüfen	Passende, vom Herausgeber signierte Lizenz für die Modul-ID
Kundenpaket veröffentlichen	Freigabe und signierter Modulkatalog des Herausgebers

Der Header allein enthält kein lauffähiges Club Platform. Bunker Development muss dem externen Entwickler einen passenden Testhost als Binärpaket, eine Testlizenz und gegebenenfalls Testzugänge bereitstellen. Diese Binärdistribution ist nicht automatisch Bestandteil des öffentlichen Doku-Repositorys.

Native Module laufen im Hostprozess und sind vertrauenswürdiger Code, keine Sandbox. Ein Modulfehler kann den Hostprozess beschädigen. Die C-Schnittstelle vermeidet private C++-Klassen an der Grenze, ersetzt aber nicht die Prüfung von Betriebssystem, Architektur, Laufzeitbibliotheken und ABI-Kompatibilität.

2. Vorab mit dem Herausgeber abstimmen

Vor der Implementierung folgende Punkte festhalten:

1. Eindeutige Modul-ID, beispielsweise [acmenotes](#), und zugehörige Berechtigungs-/Datensatzschlüssel. Diese IDs nach Auslieferung stabil halten.
2. Unterstützte Hostversionen, Betriebssysteme und Architekturen. Für den hier beschriebenen Stand ist der Beispielbereich `>= 0.6.0` und `< 0.7.0`.
3. Fachliche Daten, Beziehungen zu Personen/Organisationen, Archivierung und Verhalten bei endgültigem Löschen.
4. Benötigte andere Module und versionierte Capabilities.
5. Testhost, Administratorzugang und Lizenz mit der eigenen Modul-ID sowie gegebenenfalls ihren Abhängigkeiten.
6. Zuständigkeit für Review, Paketprüfung, Katalogsignierung, Hosting, Updates und Support.

Eine Produktions-App mit fest eingebautem Herausgeberschlüssel akzeptiert nicht einfach eine selbstsignierte Entwicklerlizenz. Der Herausgeber stellt die Testlizenz aus oder liefert ausdrücklich eine isolierte Entwicklungsumgebung mit geeignetem Vertrauensanker. Private Herausgeber-, Aktivierungs- oder Katalogschlüssel werden nicht an externe Entwickler verteilt.

Die grafische Publisher-App bietet derzeit eine feste Liste der mitgelieferten Fachmodule. Beliebige externe Modul-IDs sind dort noch nicht frei eingebbar. Eine Lizenz für eine neue ID muss Bunker Development derzeit über den vorhandenen JSON-/CLI-Signierweg ausstellen. Das ist eine notwendige organisatorische Voraussetzung, kein SDK-Buildschritt des externen Entwicklers.

3. Öffentliches SDK beziehen

Bis diese Dokumentation in den Standardbranch integriert ist, den Doku-PR-Branch verwenden:

```
git clone --branch docs/banking-activation-placeholders \
  https://github.com/jborkenhagen74/club-platform-docs.git
cd club-platform-docs
git log -1 --oneline
```

Unter PowerShell den Clone-Befehl in eine Zeile schreiben. Nach Integration kann der Standardbranch genutzt werden. Den für die Entwicklung verwendeten Commit dokumentieren, damit SDK und Beispiel reproduzierbar bleiben.

Datei / Verzeichnis	Zweck
sdk/include/clubplatform/extension_v3.h	Aktueller Modulvertrag
sdk/include/clubplatform/extension_v2.h	Wird von V3 eingebunden; muss ebenfalls vorhanden sein
sdk/examples/v3-extension/module.cpp	Vollständiges V3-Notizbeispiel
sdk/examples/v3-extension/CMakeLists.txt	Eigenständiger Modulbuild
sdk/extension_api.h	Historisches Material; nicht Ausgangspunkt neuer V3-Module
examples/extensions/hello-extension	Historisches Beispiel; nicht mit dem V3-Einstieg verwechseln

Das vollständige öffentliche Beispiel ist direkt im Repository verfügbar: `module.cpp` und `CMakeLists.txt`. Für einen ersten Durchlauf dieses Beispiel unverändert bauen, erst anschließend die eigene Modul-ID und Fachlogik einführen.

4. Das Beispiel auf den drei Systemen bauen

Das Beispiel benötigt CMake ab 3.24, einen C++17-Compiler und nlohmann-json ab 3.7 mit CMake-Paketkonfiguration. Die interne Sprache des Moduls ist unabhängig vom C++23-Build des privaten Hosts. Qt ist für dieses Modul nicht erforderlich.

Alle folgenden Befehle starten im Stamm des öffentlichen Doku-Repositorys. Der explizite `CLUB_SDK`-Pfad zeigt auf `sdk/include`, nicht auf eine einzelne Headerdatei.

4.1 macOS

Voraussetzung: eingerichtete Xcode Command Line Tools und Homebrew.

```
brew install cmake ninja nlohmann-json
cmake -S sdk/examples/v3-extension -B build/v3-example \
  -G Ninja -DCMAKE_BUILD_TYPE=Release \
  -DCLUB_SDK="$PWD/sdk/include" \
  -DCMAKE_PREFIX_PATH="$(brew --prefix nlohmann-json)"
cmake --build build/v3-example --parallel
file build/v3-example/sample.so
nm -gU build/v3-example/sample.so
```

Das CMake-Ziel ist eine **MODULE**-Bibliothek. Das Beispiel erzeugt auch auf macOS [sample.so](#), nicht automatisch eine [.app](#) oder [.dylib](#). In der Symbolausgabe muss [clubplatform_extension_v3](#) vorhanden sein; macOS zeigt gegebenenfalls einen führenden Unterstrich.

Apple-Silicon-Module mit einer ARM-Host-App testen, Intel-Module mit einem Intel-Host. Für eine andere Architektur einen eigenen Buildbaum und passende Abhängigkeiten verwenden.

4.2 Linux

Debian-/Ubuntu-Beispiel:

```
sudo apt install build-essential cmake ninja-build nlohmann-json3-dev
cmake -S sdk/examples/v3-extension -B build/v3-example \
  -G Ninja -DCMAKE_BUILD_TYPE=Release \
  -DCLUB_SDK="$PWD/sdk/include"
cmake --build build/v3-example --parallel
file build/v3-example/sample.so
nm -D --defined-only build/v3-example/sample.so
```

Das Ergebnis ist [build/v3-example/sample.so](#). Neben der CPU-Architektur die Mindestversionen von glibc und C++-Runtime beachten. Ein auf einer neuen Distribution erzeugtes Modul kann auf einer älteren Distribution nicht ladbar sein.

4.3 Windows

Voraussetzungen: Visual-Studio-C++-Werkzeuge und Windows-SDK, CMake, vcpkg mit nlohmann-json. Bei Verwendung von Visual Studio 2026 benötigt CMake den Generator [Visual Studio 18 2026](#) und damit mindestens Version 4.2. Folgende Befehle in einer Developer PowerShell ausführen; der vcpkg-Pfad muss bereits existieren.

```
$env:VCPKG_ROOT = 'C:\Dev\vcpkg'
& "$env:VCPKG_ROOT\vcpkg.exe" install nlohmann-json:x64-windows
$sdk = (Resolve-Path .\sdk\include).Path
$toolchain = "$env:VCPKG_ROOT\scripts\buildsystems\vcpkg.cmake"
cmake -S sdk/examples/v3-extension -B build/v3-example `
  -G 'Visual Studio 18 2026' -A x64 `
  -DCLUB_SDK=$sdk -DCMAKE_TOOLCHAIN_FILE=$toolchain `
  -DVCPKG_TARGET_TRIPLET=x64-windows -DVCPKG_MANIFEST_MODE=OFF
cmake --build build/v3-example --config Release --parallel
Get-Item .\build\v3-example\Release\sample.dll
dumpbin /exports .\build\v3-example\Release\sample.dll
```

Hier wird nlohmann-json im klassischen vcpkg-Modus als reine Headerbibliothek verwendet. Für eigene zusätzliche Bibliotheken eine bewusste und reproduzierbare Toolchain-/Manifestkonfiguration anlegen. Der private Host-Build verwendet einen anderen vcpkg-Triplet; es werden keine privaten C++-Objekte zwischen Modul und Host ausgetauscht. Trotzdem müssen eigene Laufzeitabhängigkeiten zum Zielsystem passen.

4.4 Erwartetes Ergebnis

Der Build erzeugt eine native Bibliothek, keinen Club-Platform-Installer und keine Lizenz. Für jede angebotene Kombination aus Betriebssystem und CPU ist ein eigenes geprüftes Artefakt erforderlich. Eine Windows-DLL wird nicht auf macOS geladen; eine ARM-Bibliothek nicht in einem x64-Prozess.

Fehlt [nlohmann_jsonConfig.cmake](#), das Entwicklungspaket mit CMake-Konfiguration installieren oder dessen Prefix über [CMAKE_PREFIX_PATH](#) angeben. Fehlt [extension_v2.h](#), wurde nur ein Teil des SDK kopiert. Bei einem Generatorwechsel einen neuen Buildbaum verwenden.

5. ABI V3 verstehen

Die zentrale Struktur aus dem öffentlichen Header lautet:

```
typedef struct ClubExtensionV3 {
    uint32_t abi_version;
    uint32_t struct_size;
    const char* manifest_json;
    int (*validate)(const char* record_type, const char* values_json);
    int (*lifecycle)(const char* event, const char* context_json);
    const char* (*migrate_record)(uint32_t version,
        const char* record_type, const char* values_json);
    const char* (*invoke)(const char* capability,
        uint32_t version, const char* input_json);
} ClubExtensionV3;
```

Das Modul exportiert genau den vereinbarten Einstieg. Beispielstruktur, deren benannte Variablen und Funktionen aus der eigenen Implementierung stammen müssen:

```
const ClubExtensionV3 api{
    3, static_cast<uint32_t>(sizeof(ClubExtensionV3)),
    manifest.c_str(), validate, lifecycle, migrate, invoke
};
extern "C" CLUB_EXTENSION_EXPORT
const ClubExtensionV3* clubplatform_extension_v3() {
    return &api;
}
```

api und der Speicher für **manifest** müssen über die gesamte Ladedauer gültig bleiben. Die Initialisierung darf keine Ausnahme über die Modulgrenze tragen. Der Host prüft ABI-Version, Strukturgröße und sämtliche Callbackzeiger. Ein ungültiger V3-Export wird nicht durch einen zusätzlich vorhandenen V2-Export repariert: V2 wird nur verwendet, wenn kein V3-Symbol existiert.

Callback	Aufgabe	Rückgabe
validate	Datensatzwerte fachlich prüfen	1 gültig, 0 fachlich abgelehnt, -1 Fehler
lifecycle	Hostereignis verarbeiten	1 Erfolg, 0 Ablehnung, -1 Fehler
migrate_record	Werte eines bestehenden Datensatzes transformieren	UTF-8-JSON oder <code>nullptr</code> bei Fehler
invoke	Versionierte Capability bedienen	UTF-8-JSON oder <code>nullptr</code> bei Fehler

Für alle Callbacks gilt:

- Eingabezeiger gehören dem Host und dürfen nicht für spätere Aufrufe gespeichert werden.
- Rückgabe-JSON gehört dem Modul und bleibt bis zum nächsten Aufruf gültig. Der Host kopiert es sofort. Keine Zeiger auf lokale Stackvariablen zurückgeben.
- Keine Exceptions, STL-Container, C++-Objekte, Datenbankhandles oder fremd freizugebenden Speicher über die ABI-Grenze geben.
- Aufrufe werden serialisiert. Eigene Hintergrundthreads dürfen daraus keine zusätzlichen unkoordinierten Zugriffe ableiten.
- Keine externen Seiteneffekte in Callbacks. Eine Datenbanktransaktion des Hosts kann externe Netzwerkaktionen oder fremde Dateischreibzugriffe nicht rückgängig machen.
- JSON strikt prüfen: Typen, Pflichtfelder, Grenzen und fachliche Werte. Ungültige Eingaben kontrolliert zurückweisen.

Das öffentliche Beispiel fängt Parsingfehler ab und verwendet für JSON-Rückgaben einen Modul-eigenen `std::string`. Für ein Produktionsmodul zusätzlich Datensatztyp und Capability explizit prüfen. Unbekannte Typen nicht versehentlich als eigenen Datensatz akzeptieren.

6. Eigenes Manifest und eigene Datensätze

Das Beispiel deklariert das Modul `sample` mit dem Typ `sample.note` und dem Textfeld `text`. Für ein eigenes Modul die Datei zunächst in ein eigenes Projekt übernehmen. Modul-ID, CMake-Zielname, Typenschlüssel, Berechtigungen und Übersetzungsschlüssel konsistent umbenennen. Bereits veröffentlichte Identitäten nicht nachträglich wechseln.

Ein minimales fachliches Manifest kann so aussehen; es muss als `manifest_json` des Moduls ausgegeben werden und ersetzt nicht die Callback-Implementierung:

```
{
  "id": "acmenotes",
  "name_key": "module.acmenotes.name",
  "version": "1.0.0",
  "abi": 3,
  "core": {
    "minimum": "0.6.0",
    "maximum_exclusive": "0.7.0"
  },
  "schema_version": 1,
  "migrations": [
    {"version": 1, "definition": "Initial note records"}
  ],
  "permissions": [
    "acmenotes.read", "acmenotes.write", "acmenotes.archive",
    "acmenotes.restore", "acmenotes.delete"
  ],
  "capabilities": [
    {"key": "record.references", "version": 1}
  ],
  "types": [{
    "key": "acmenotes.note",
    "label": "Notiz",
    "label_key": "module.acmenotes.note",
    "fields": [{
      "key": "text", "label": "Text", "type": "text", "required": true
    }]
  }]
}
```

Dieses Manifest bietet bewusst nur `record.references` an. Wird zusätzlich die Dokument-Capability des öffentlichen Beispiels übernommen, müssen auch deren Deklaration, Platzhaltermetadaten und Implementierung zusammenpassen. Umgekehrt keine Capability deklarieren, deren Aufrufe stets fehlschlagen.

Feld	Bedeutung
id	Stabile Modulidentität, auch für Lizenz und Paketkatalog
version	Numerisches Tripel, beispielsweise 1.0.0; kein Prerelease-Suffix
core.minimum	Kleinste akzeptierte Hostversion, einschließlich
core.maximum_exclusive	Obere Hostgrenze, ausschließlich
schema_version	Version der Moduldatensatzstruktur
migrations	Lückenlose Historie mit unveränderlichen Definitionen
permissions	Berechtigungen des Moduls

Feld	Bedeutung
types	Hostverwaltete Datensatztypen und ihre Felder
capabilities	Versionierte Funktionen, die das Modul anbietet
dependencies	Optionale Liste benötigter Module und Versionsbereiche
requires	Benötigte Capability eines anderen Moduls

Beispiel einer Modulabhängigkeit: `{"id":"finance","minimum":"1.1.0"}` in `dependencies`. Eine Capability-Abhängigkeit in `requires` verwendet die Felder `module`, `key` und `version`. Nur tatsächlich im Zielhost vorhandene Verträge verwenden; einen Namen nicht aus einem geplanten Feature ableiten.

Eigene Berechtigungen müssen anschließend geeigneten Benutzerrollen zugeordnet werden. Ein im Manifest aufgeführtes Schreibrecht erteilt dem angemeldeten Benutzer nicht automatisch diese Berechtigung. Installation/Lizenzverwaltung erfordert zusätzlich das administrative Recht `schema.manage`.

7. Oberfläche, Daten und Grenzen der Erweiterbarkeit

Die Hostoberflächen verwenden deklarierte Modultypen und Metadaten. Das V3-Notizbeispiel nutzt hostverwaltete Datensätze. Ein Modul bekommt über ABI V3 keinen privaten Datenbankzugang und keinen allgemeinen Host-API-Zeiger zum freien Registrieren von Menüs oder REST-Routen.

Die historischen SDK-Dokumente beschreiben teilweise frühere oder geplante UI-Beiträge. Aus ihnen darf nicht abgeleitet werden, dass eine beliebige eigene Qt-Ansicht, ein eigener HTTP-Endpunkt oder ein Hintergrundjob allein durch einen V3-Export verfügbar wird. Für eine Funktion außerhalb der veröffentlichten Verträge ist eine abgestimmte Host-/SDK-Erweiterung erforderlich.

Übersetzungsschlüssel wie `name_key` und `label_key` sind stabile Identitäten, keine automatisch erzeugten Übersetzungen. Sichtbare Fallback-Labels sinnvoll setzen und die Bereitstellung der Übersetzungen mit dem Herausgeber abstimmen. Funktionsnamen, Berechtigungsschlüssel und Datensatz-IDs werden nicht lokalisiert.

Die Modulbibliothek erhält Datensatzwerte als JSON. Persistenz, IDs und Transaktionsgrenzen bleiben beim Host. Keine eigene SQL-Migration gegen vermeintliche interne Tabellen schreiben: Die Tabellenstruktur gehört nicht zum öffentlichen ABI-Vertrag.

8. Lifecycle und migrationssichere Updates

Lifecycle-Ereignisse sind `load`, `install`, `upgrade`, `enable` und `disable`. Deaktivieren entlädt die native Bibliothek nicht. Für eine neue native Bibliothek ist ein Neustart des zuständigen Hosts erforderlich.

Beispiel: Version 1.0.0 kennt nur `text`; Version 1.1.0 ergänzt `category`.

1. Modulversion auf `1.1.0` und `schema_version` auf 2 erhöhen.
2. Migration 1 unverändert behalten und Migration 2 mit stabiler Definition ergänzen.
3. Feld `category` im Manifest deklarieren.
4. In `migrate_record` für Schritt 2 vorhandene Werte lesen und `category` mit einem geeigneten Standardwert ergänzen.
5. Unbekannte oder unzulässige Eingaben kontrolliert ablehnen. Bestehende Felder und Werte nicht unbeabsichtigt verlieren.
6. Upgrade mit echten Testdatensätzen und zusätzlich mit absichtlich fehlerhafter Migration testen.

Der Host führt Installation/Migration einschließlich Manifest und Audit innerhalb einer Datenbanktransaktion durch. Datensatz-IDs bleiben erhalten. Ein Fehler muss die gesamte betroffene Migration zurückrollen. Das schützt nicht vor externen Seiteneffekten des Moduls; diese sind in den Callbacks deshalb zu vermeiden.

Downgrades, Lücken in der Migrationshistorie und Änderungen bereits gespeicherter Migrationsdefinitionen werden abgewiesen. Einen Rollback nicht durch einfaches Zurückkopieren einer alten Bibliothek versuchen. Vor einem Update Datenbank und zusammengehörigen Modulstand sichern und eine Wiederherstellung auf dem Testsystem erproben.

Das Beispiel enthält `accepts_v2: true`, weil es auch V2-Übernahmetests dient. Für ein neues, ausschließlich V3-basiertes Produkt diese Zusage nicht unüberlegt übernehmen. V2-Kompatibilität nur deklarieren, wenn genau diese Datenübernahme implementiert und getestet ist. Die Makrovarianten `V3_FIXTURE` sind Regressionstestfälle, keine auszuliefernden Editionen.

9. Referenzen und Dokumentplatzhalter

9.1 Endgültiges Löschen sicher prüfen

Die Capability `record.references` in Version 1 erhält eine Ziel-ID und die Moduldatensätze einschließlich archivierter Einträge. Sie antwortet beispielsweise mit `{"referenced":false}`. Das öffentliche Beispiel prüft dazu `owner_id` gegen `entity_id`.

Die eigene Implementierung muss alle tatsächlich gespeicherten Beziehungen berücksichtigen, einschließlich eigener Referenzfelder und archivierter Daten. Nicht pauschal `false` zurückgeben, nur damit die Löschaktion funktioniert. Fehlende, deaktivierte oder nicht eindeutig antwortende Module können endgültiges Löschen blockieren; Archivierung bleibt der sichere reguläre Weg.

9.2 Dokumente erweitern

Das öffentliche Notizbeispiel deklariert `document.placeholder-provider` in Version 1 und den Platzhalter `sample.note.text`. Die Metadaten beschreiben unter anderem Scope, Werttyp, benötigte Berechtigung und zugehörigen Datensatztyp. `invoke` liefert für diese Capability ein JSON-Objekt mit `value`.

Die Demo wählt den ersten übergebenen Datensatz. Für ein produktives Modul ausdrücklich definieren, wie kein Datensatz, mehrere Datensätze, fehlende Leserechte und unbekannte Werte behandelt werden. Die Dokumentengine darf durch einen Platzhalter keine unberechtigten Daten offenlegen. Den vollständigen Vertrag in der Dokumentreferenz prüfen.

10. Integration mit einer binären Testinstallation

10.1 Getrennte Testumgebung vorbereiten

1. Vom Herausgeber einen zur Architektur passenden Testhost und eine Testlizenz für `sample` beziehungsweise die eigene ID erhalten.
2. Eine neue Testdatenbank verwenden. Produktivdatenbanken und produktive Modulverzeichnisse nicht für Entwicklungsversuche nutzen.
3. Die Bibliothek in ein eigenes Modulverzeichnis kopieren. Falls andere Module benötigt werden, auch deren passende Binärdateien und Lizenzen bereitstellen.
4. Host mit diesem Verzeichnis starten und Moduldiagnosen prüfen.
5. Als Testadministrator anmelden, Lizenz importieren und bei einer gebundenen Lizenz die Installation aktivieren.
6. Modul installieren/aktualisieren, Berechtigungen zuweisen und Fachfunktionen testen.

Beispiel macOS nach Installation einer bereitgestellten Desktop-App; weiterhin im öffentlichen Doku-Checkout:

```
mkdir -p "$HOME/club-module-test/extensions"
cp build/v3-example/sample.so "$HOME/club-module-test/extensions/"
open -n /Applications/ClubPlatform/clubplatform-desktop.app \
  --args --database "$HOME/club-module-test/test.sqlite" \
  --extensions "$HOME/club-module-test/extensions"
```

Linux mit bereitgestelltem Desktoppaket:

```
mkdir -p "$HOME/club-module-test/extensions"
cp build/v3-example/sample.so "$HOME/club-module-test/extensions/"
/opt/club-platform-desktop/bin/clubplatform-desktop \
  --database "$HOME/club-module-test/test.sqlite" \
  --extensions "$HOME/club-module-test/extensions"
```

Unter Windows den tatsächlich installierten App-Pfad abfragen, statt ein bestimmtes Installationslaufwerk vorauszusetzen:

```
$testRoot = Join-Path $HOME 'club-module-test'
New-Item -ItemType Directory -Force "$testRoot\extensions"
Copy-Item .\build\v3-example\Release\sample.dll "$testRoot\extensions\"
$app = Read-Host 'Vollstaendiger Pfad zur clubplatform-desktop.exe'
& $app --database "$testRoot\test.sqlite" `
  --extensions "$testRoot\extensions"
```

10.2 Server statt lokalem Desktop

Bei einem Remote-Desktop oder Portal liegt das Modul auf dem Server. Dessen `--extensions`-Verzeichnis und Datenbank sind maßgeblich. Das Kopieren einer DLL oder `.so` auf den Client erweitert den Server nicht.

Für einen bereitgestellten Server denselben isolierten Ansatz verwenden: neue SQLite-Datei, zunächst `--init admin`, dann normaler Serverstart mit `--sqlite`, `--extensions` und einem freien Port. Die Produkt-Buildanleitung enthält ein Server-Testbeispiel. Externe Entwickler benötigen dafür nur das Serverbinary, nicht dessen Quellcode.

10.3 Gefunden ist nicht gleich betriebsbereit

Die Anzeige unterscheidet gefunden, geladen, installiert und lizenziert. Ein erfolgreich exportiertes Symbol allein macht das Modul noch nicht nutzbar. Manifestfehler, fehlende Abhängigkeiten, Lizenz oder Aktivierung sowie Benutzerrechte unabhängig prüfen.

Nach jeder Änderung der nativen Bibliothek den Host vollständig beenden und neu starten. Bei einem schon über den Modulkatalog verwalteten Host kann die aktive installierte Generation Vorrang vor einem frei kopierten Modul haben. Für direkte Entwicklertests eine frische isolierte Umgebung verwenden; für Produktion den Katalogupdateweg nutzen.

11. Prüfplan für den Entwickler

Bereich	Erwartete Prüfung
ABI	Symbol sichtbar, Version 3, korrekte Strukturgröße, alle Callbacks gesetzt
Validierung	Gültig, leer, falscher JSON-Typ, fehlendes Feld, unbekannter Datensatztyp
Speicher	Keine hängenden Eingabezeiger, gültige Rückgabepuffer, keine Exceptions über ABI
Manifest	Eindeutige IDs, passende Hostversion, deklarierte Funktionen tatsächlich implementiert
Rechte	Lesen/Schreiben ohne Recht abgewiesen; berechtigter Benutzer erfolgreich
Lizenz	Eigene Modul-ID und Abhängigkeiten lizenziert; Sperr-/Ablaufverhalten geprüft
Lifecycle	Erstinstallation, wiederholte Installation, Aktivieren/Deaktivieren, Neustart
Migration	Daten erhalten, neue Felder korrekt, Fehlerfall rollt zurück
Beziehungen	Archivierte und aktive Referenzen verhindern unsicheres Löschen
Dokumente	Kein Wert, mehrere Werte, Berechtigungen, Übersetzungen
Plattform	Saubere Zielmaschine, passende CPU, fehlende Bibliotheken erkannt
Update	Neuer Katalogstand, Neustart, Backup und Wiederherstellung

Die reine Callback-Logik in eigenen Tests ausführen. Für das Laden, die Hosttransaktionen, Lizenzierung und Oberflächen sind Integrationstests mit dem bereitgestellten Host nötig. Keine vollständige Plattformfreigabe aus einem einzigen Linux-Build ableiten.

Vor der Übergabe protokollieren: SDK-Commit, Hostversion, Modulversion, Schema-Version, Compiler und Betriebssystem/Architektur, Testfälle und Ergebnisse. Einen reproduzierbaren Build bereitstellen. Debugpfade, Testschlüssel, Zugangsdaten und personenbezogene Testdaten dürfen nicht im Modul landen.

12. Auslieferung über den signierten Modulkatalog

Ein Entwickler liefert die geprüfte native Bibliothek und die zugehörigen Metadaten an Bunker Development. Der aktuelle Paketvertrag behandelt die einzelne Bibliotheksdatei als Nutzlast, nicht ein beliebiges ZIP mit Installationsskripten. Zusätzliche native Abhängigkeiten müssen statisch eingebunden oder auf dem Zielsystem ausdrücklich bereitgestellt sein. Keine versteckten Downloads bei Modulaufrufen einbauen.

Metadatum	Inhalt
<code>module_id</code> , <code>version</code>	Müssen zum exportierten Manifest passen
<code>platform</code>	<code>windows</code> , <code>macos</code> oder <code>linux</code>
<code>architecture</code>	Katalogkennung <code>x64</code> oder <code>arm64</code>
<code>core_range</code>	Unterstützte Hostversionen mit <code>minimum</code> und optional <code>maximum_exclusive</code>
<code>released_at</code>	Veröffentlichungszeit als UTC-Unixsekunden

Metadatum	Inhalt
download_location	Zulässige HTTPS-Adresse oder freigegebener Offline-Dateiverweis
sha256	SHA-256 der endgültigen Bibliotheksdatei
dependencies	Passend zum Manifest und den tatsächlich benötigten Modulen
manifest	Vollständiges vom Modul exportiertes Manifest
signature	Vom freigebenden Herausgeber erzeugte Paketsignatur

Die Architekturkennung im Modulkatalog ist nicht zwingend derselbe Text wie der Architekturteil im Namen eines Produktinstallers. Dateien nach ihrer tatsächlichen Architektur klassifizieren, nicht nach einem ungeprüften Dateinamen.

Der Herausgeber prüft das Paket, signiert den Deskriptor und erstellt einen signierten Katalog mit [issued_at](#), [expires_at](#) und [packages](#). Lizenzsignatur, Aktivierung und Katalogsignatur haben getrennte Aufgaben. Das vollständige Format und die Signierwerkzeuge beschreibt die Provisionierungsreferenz.

Das Herausgeberwerkzeug [clubplatform-sign manifests](#) lädt die Bibliotheken, um deren Manifest auszulesen. Das ist keine gefahrlose Textinspektion; nur bereits geprüfte, vertrauenswürdige Binärdateien in einer geeigneten Prüfungsumgebung laden.

Kundenseitig erfolgt die Installation über **Module aus Katalog installieren**, anschließend der angeforderte Neustart des lokalen Hosts oder Servers. Ein gültiger Katalog ersetzt keine Modullizenz. Auch Abhängigkeiten müssen freigeschaltet sein. Das Wartungsende begrenzt verfügbare neue Paketversionen anhand ihres Veröffentlichungszeitpunkts; es ist vom Lizenzablauf zu unterscheiden.

Für Offline-Pakete die dokumentierte Offline-Verzeichnisfunktion verwenden. Der Dateiverweis zeigt auf die einzelne zugelassene Datei, nicht auf einen beliebigen Dateisystempfad. Abgelaufene Kataloge können Neuinstallationen blockieren; sie löschen keine bereits installierten Moduldatensätze.

13. Fehlerdiagnose

Symptom / Diagnose	Typische Ursache und Maßnahme
Modul nicht gefunden	Verzeichnis, Dateieindung und tatsächlich laufenden Host prüfen
Bibliothek nicht ladbar	Falsche CPU/OS, fehlende Runtime oder abhängige Bibliothek
V3-Deskriptor ungültig	Symbol, <code>abi_version</code> , <code>struct_size</code> , Lebensdauer und Callbackzeiger prüfen
<code>dependency_missing</code>	Benötigtes Modul fehlt oder passt nicht zur Deklaration
<code>dependency_cycle</code>	Zyklische Modulabhängigkeit auflösen
<code>version_mismatch</code>	Host-/Modulversionsbereich stimmt nicht
<code>capability_missing</code>	Angeforderte Funktion wird vom Provider nicht angeboten
<code>capability_version_mismatch</code>	Angeforderte und unterstützte Vertragsversion unterschiedlich
<code>lifecycle_failed</code>	Callback gibt Ablehnung/Fehler zurück; Eingaben und Moduldiagnose prüfen
Nicht lizenziert	Eigene Modul-ID oder Abhängigkeit fehlt in der wirksamen Lizenz
Keine Berechtigung	Benutzerrechte prüfen; gültige Lizenz ist keine Rechtezuweisung
Alte Bibliothek weiterhin geladen	Hostprozess nicht neu gestartet oder aktive Kataloggeneration verwendet
Migration abgewiesen	Historie verändert, Version ausgelassen, Downgrade oder ungültige Datentransformation
Endgültiges Löschen gesperrt	Referenzen vorhanden oder Capability kann Sicherheit nicht bestätigen

Für Support immer Modulmanifest, Host-/SDK-Version, Plattform, genaue Diagnose und reproduzierbare Schritte angeben. Keine privaten Signierschlüssel oder produktiven personenbezogenen Daten übermitteln.

14. Übergabe an Bunker Development

Zur Freigabe gehören pro Zielplattform die Bibliothek, ihr SHA-256, das exportierte Manifest, Abhängigkeits-/Runtimeangaben, Buildanleitung und Testprotokoll. Zusätzlich Änderungen an Datensätzen, Rechtekonzept, Dokumentplatzhaltern, Migrationen und Wiederherstellung beschreiben. Die eigene Modul-ID, Testlizenz und Katalogaufnahme vor dem Kundeneinsatz abstimmen.

Die Markdown-Datei ist die bearbeitbare Quelle dieser Anleitung. PDF-Erzeugung und Pflege sind in `docs/pdf/README.md` dokumentiert. Bei Änderungen am öffentlichen SDK müssen Beispiel, Anleitung und PDF zusammen aktualisiert werden.